Concepção e Autoria Multimédia

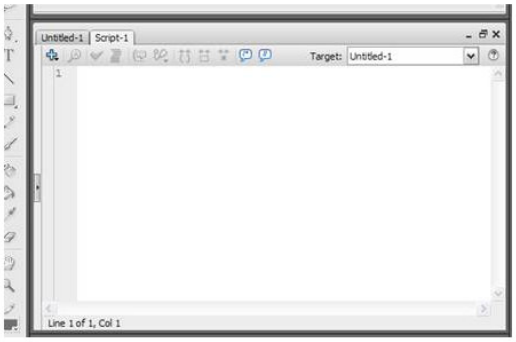
Carlos Vaz de Carvalho

New characteristics

- Fully Object Oriented Language
- New virtual machine – AVM2
  - 10x playback performance compared to AVM1 because it was designed specifically for AS 3.0
- New data types - int and uint – to use faster integer-based math for calculations
- New display list model – a new, more flexible approach to managing display objects to be viewed on the screen
- New event model – a new, listener-based event model with support for event propagation

New features

- Runtime exceptions – errors that are thrown at runtime (during SWF playback) to help debug your project
- Runtime variable typing – typing which goes beyond compilation and persists during playback
- E4X – a new, easy to work with implementation of XML
- Namespaces – support for namespaces in class definitions to set custom access to class members
- Includes an actionscript editor (not only an Actions window)



Basic variable handling

- *// int, unit, Number and Boolean types cannot be deleted*
  var intNum:int = 0;
  var uintNum:uint = 0;
  var numNum:Number = NaN;
  var bool:Boolean = false;

- *// Create a dynamic object with dynamic property*
- *// A dynamic property does not belong to the class declaration*
  var obj:Object = new Object();
  obj.prop = "value";

- *// delete dynamic property on obj using delete*
  delete obj.prop
-
- *// cannot delete obj, only able to set to null*
  obj = null;

Functions

- *// Named function* – usual in AS 2.0

```
function doubleNumber(num:Number):Number
    { return num *2;}
```

- *// Same function defined as an anonymous function*

```
var doubleNumber:Function =
    function(num:Number):Number { return num*2; }
```

- If a function does not return a value, then its return type should be void.

```
function returnsNothing():void { // do nothing }
trace(returnsNothing()); // traces undefined
```

Arrays and Dictionaries

- *// Main difference: only Dictionary*
- *// instances can have object references as*
- *// index keys*

```
var arr:Array = new Array();
arr[0] = "value";

var obj:Object = new Object();
obj["variable"] = "value";

var dict:Dictionary = new Dictionary();
dict[arr] = "value1";
dict[obj] = "value2";
```

New visual symbol classes

- Sprite (flash.display.Sprite) – like a MovieClip but with no frames
  - *Sprites* are a new kind of light-weight MovieClip that doesn't technically have any frames. In a more practical, Flash-like sense, you can see a Sprite as having one frame in which it keeps its content or children, but there is no navigation to any frames beyond that first. Since MovieClip instances you create dynamically through ActionScript cannot have frames anyway, chances are you'd want to use a Sprite in place of a MovieClip for all dynamically created MovieClip instances.
- Shape (flash.display.Shape) – like a Sprite but with no mouse or keyboard interactivity
  - *Shapes* are like Sprites but are impeded by the fact that they offer no form of mouse or keyboard interactivity. In other words, these display objects cannot receive events like mouseDown or keyUp. Additionally, Shape instances cannot contain any other kind of display objects nor have children of their own. You can only draw vector shapes within them using the vector drawing API (beginFill, lineTo, endFill, etc). This comes as an advatage to Shapes since they use less memory and are more efficient than MovieClips and Sprites.

New visual symbol classes

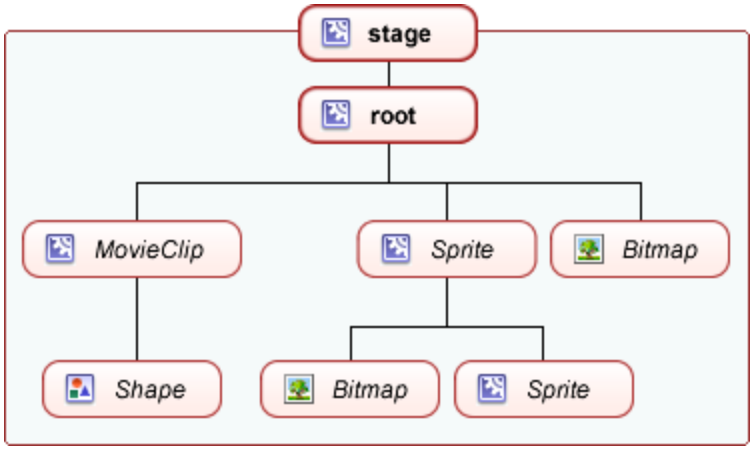
- Bitmap (flash.display.Bitmap) – like a Sprite but used only for displaying bitmap images
  - *Bitmap* instances are much like Shapes except instead of being able to contain vector drawings, they contain bitmap images defined by a BitmapData instance. They, too, cannot contain any other display objects or children. Unlike Shapes, Bitmaps can interact with the mouse and keyboard.
- Loader (flash.display.Loader) – like a Sprite but used specifically to display and contain loaded content (SWFs and images)
  - *Loader* is a Sprite-like display object which, like Sprite, has no timeline, but is designed specifically to facilitate the loading of external display content such as external SWFs and bitmap images. When loaded, those assets displayed in the Loader instance and are accessible through the Loader's content property.
- Class inheritance

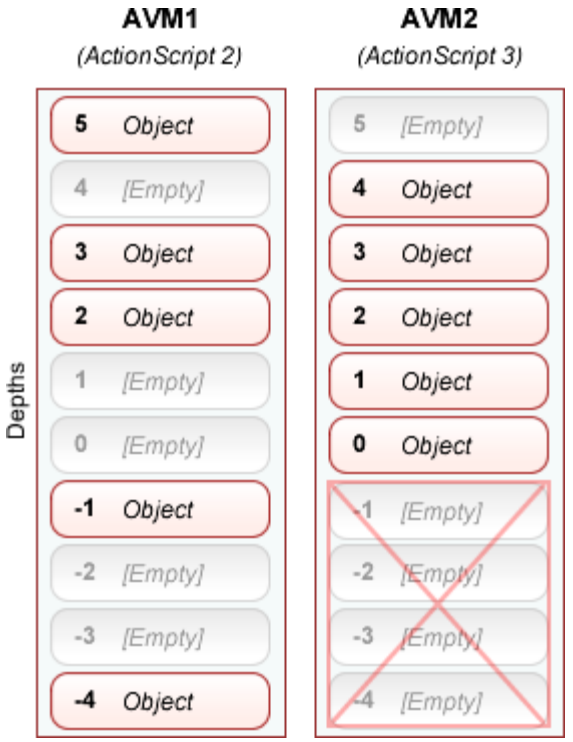
MovieClip > Sprite > DisplayObjectContainer > InteractiveObject > DisplayObject > EventDispatcher > Object

New object properties names

Common MovieClip Property Names	
ActionScript 2	**ActionScript 3**
_name	name
_x	x
_y	y
_width	width
_height	height
_xscale (0-100)	scaleX (0-1)
_yscale (0-100)	scaleY (0-1)
_rotation	rotation
_alpha (0-100)	alpha (0-1)
_visible	visible
_xmouse	mouseX
_ymouse	mouseY
_parent	parent

New display list model





Basic object handling

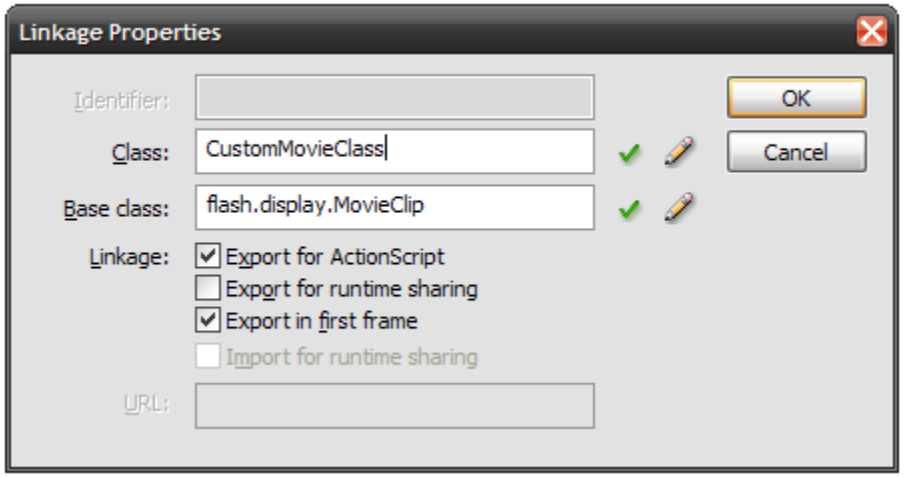
- *// Create a new, dynamic, Sprite and add it*
- *// to the current timeline's display list*
- *// AS3 equivalent to createEmptyMovieClip*

```
var mySprite:Sprite = new Sprite();
addChild(mySprite);
```

Basic object handling

```
// Create a new, dynamic movie clip from the
//library and add it to the timeline's display list
// ActionScript 3 equivalent to attachMovie
// Start by linking movie clip (linkage) to a class

var myMovie:CustomMovieClass = new
    CustomMovieClass();
addChild(myMovie);
```




```
// Remove movie clip
// from display list
removeChild(myMovie);
```

Basic bitmap handling

- *// Create an instance of a library bitmap given the Class name of MyImage*

  var imageData:MyImage = new MyImage(0, 0);

- *// Give the instance to a Bitmap through its*
- *// constructor, then add it to the display list*

  var imageBmp:Bitmap = new Bitmap(imageData);

  addChild(imageBmp);

Basic vector drawing

- The Graphics class is now used to facilitate all dynamic drawings. Instances of this class are available as graphics properties within the classes that allow you to draw in them dynamically.
- These classes include: Shape, Sprite, MovieClip

- *// Create a blue triangle with a red*
- *// outline within the current timeline*

```
graphics.lineStyle(2, 0xFF0000);
graphics.beginFill(0x0000FF);
graphics.moveTo(50, 0);
graphics.lineTo(100, 100);
graphics.lineTo(0, 100);
graphics.endFill();
```

New vector drawing methods

- In addition to the methods available in ActionScript 2, AS 3 adds some new methods to make drawing simple shapes like rectangles and circles a lot easier. These methods include:
  - drawCircle
  - drawEllipse
  - drawRect
  - drawRoundRect
  - drawRoundRectComplex

New event handler philosophy

- Event handling in ActionScript 3 means:
  - using addEventListener/removeEventListener
  - listening for event types stored in Event classes
  - using a function (not an object) event handler/listener
  - defining event handler/listeners to receive an argument of the type Event

Event list

Common Interactive Events		
ActionScript 2	**ActionScript 3**	**Differences**
onPress	"mouseDown" (MouseEvent.MOUSE_DOWN)	
onRelease	"mouseUp" (MouseEvent.MOUSE_UP), "click" (MouseEvent.CLICK)	The click event is most like onRelease since it depends on the object first receiving a mouseDown event whereas mouseUp can occur regardless of what was pressed
onReleaseOutside	"mouseUp" (MouseEvent.MOUSE_UP)	This has been removed, but using the mouseUp event with the stage (because of propagation) will allow you to detect when the mouse has been released outside of the object originally pressed
onRollOver	"mouseOver" (MouseEvent.MOUSE_OVER), "rollOver" (MouseEvent.ROLL_OVER)	Two events can now be used for rolling over objects; mouseOver will propagate, rollOver will not
onRollOut	"mouseOut" (MouseEvent.MOUSE_OUT), "rollOut" (MouseEvent.ROLL_OUT)	Two events can now be used for rolling off of objects; mouseOut will propagate, rollOut will not
onDragOver	"mouseOver" (MouseEvent.MOUSE_OVER), "rollOver" (MouseEvent.ROLL_OVER)	Detecting a "drag" will have to be done manually in combination with the mouseDown event.
onDragOut	"mouseOut" (MouseEvent.MOUSE_OUT), "rollOut" (MouseEvent.ROLL_OUT)	Detecting a "drag" will have to be done manually in combination with the mouseDown event.
mouseDown	"mouseDown" (MouseEvent.MOUSE_DOWN)	Now only works when the mouse is over the object dispatching the event (like onPress); normal objects cannot be made listeners of the Mouse to be able to recieve this event, only InteractiveObject instances can
mouseUp	"mouseUp" (MouseEvent.MOUSE_UP)	Now only works when the mouse is over the object dispatching the event (like onRelease but not requiring that the object be clicked first); normal objects cannot be made listeners of the Mouse to be able to recieve this event, only InteractiveObject instances can
mouseMove	"mouseMove" (MouseEvent.MOUSE_MOVE)	Now only works when the mouse is over the object dispatching the event; normal objects cannot be made listeners of the Mouse to be able to recieve this event, only InteractiveObject instances can
onKeyDown	"keyDown" (KeyboardEvent.KEY_DOWN)	Object must now have focus to receive this event; normal objects cannot be made listeners of the Key to be able to recieve this event, only InteractiveObject instances can
onKeyUp	"keyUp" (KeyboardEvent.KEY_UP)	Object must now have focus to receive this event; normal objects cannot be made listeners of the Key to be able to recieve this event, only InteractiveObject instances can

- http://moodle.isep.ipp.pt
- cmc@isep.ipp.pt